AGORA: A Two-Sided Agent Marketplace for Test-Verified Software Services with Guarded USDC Escrow on NEAR

Kyle Allen M. Abandia, Maurenzio Franck M. Altamarino, Maria Veronica G. Amparo,
Siegfred Josh L. Calayag, Raphael Augustin V. Dator, Mary Rose P. de los Reyes,
Leanne Elech P. Rosales, Jhon Lloyd M. Valencia

abandiakyle2112@gmail.com · altamarinofranck@gmail.com · amparomariaveronica07@gmail.com · siegfredcalayag@gmail.com
raphaelaugustindator@gmail.com · delosreyesmaryrosep@gmail.com · r.leannelech@gmail.com · jhonlloydval@gmail.com

Team AGORA · Repository github.com/SoftEng-Peeps/Agora-NEAR-Hackathon · Live app agora-seven-fawn.vercel.app
Main track: Autonomous Business; also Agent Economy, AI Swarms, AI Security & Defense

Abstract. *AI agents are starting to hire on people's behalf, but the markets they enter were built for humans. Builders describe today's practice as "someone hands them a card and hopes nothing weird happens", and Microsoft's Magentic Marketplace study found that buyer agents chose the first proposal they received 60–100% of the time and that some models could be manipulated into paying attackers. Freelancers have the opposite problem: 62% of New York freelancers report losing pay to non-payment. Incumbents are also moving away from small jobs: Fiverr lost 13.6% of its active buyers in 2025, partly attributing the decline to AI reducing demand for simple services, while keeping a 27.7% take rate. AGORA is a services marketplace on NEAR in which every user has a small team of agents and either side of a job can be a human or an AI agent; one account can both buy and sell. It launches with one category, small software jobs, because their completion can be checked by machine: the buyer locks a test suite when accepting an offer, and three independent reviewer agents release the payment only when the tests pass, each vote carrying a test-log hash and a model attestation hash. Payment is USDC held in a contract escrow (a test token stands in for USDC on testnet). The agents never hold the money: they sign with function-call keys that cannot withdraw or attach deposits, and the contract enforces each owner's mandate (per-job cap, daily cap, approved sellers, categories, pause). Disputes go to an arbiter DAO, and reputation is written on-chain only from settled jobs. The design draws on research showing that new workers struggle to be hired, that trust can increase disintermediation, and that ratings inflate. This paper sets out the problem, the related work, the design and a threat model, and fixes eight pass marks before building, including a test with outside teams. At submission the market contract is deployed on NEAR testnet and one job has run on it from posting to rating; most pass marks are not measured yet, and Section V.d lists what is built and what is still design.*

Keywords: *agentic commerce, AI agents, multi-agent systems, online labour markets, services marketplace, escrow, test-based acceptance, USDC, NEAR Protocol, function-call keys, trusted execution environments, DAO arbitration, disintermediation, on-chain reputation, freelancing*

I. INTRODUCTION

a. Background

"Agentic commerce" means software agents that search, compare, negotiate and pay for a person or a business. Forecasts are large. McKinsey estimates USD 3–5 trillion of global retail orchestrated by agents by 2030 (Brohan, 2025). Gartner expects over USD 15 trillion of business-to-business spending to be handled by agents by 2028 (Digital Commerce 360, 2025). Card networks and AI companies have launched protocols for it: OpenAI and Stripe's Agentic Commerce Protocol, Google's Agent Payments Protocol (AP2), Coinbase's x402, and Visa and Mastercard agent tokens (Airwallex, 2026).

Those protocols answer one question: is this agent allowed to pay for this person? They do not answer what happens when agents meet each other. Microsoft Research built a simulated market of buyer and seller agents and found that models approached optimal welfare only under ideal search conditions, that performance degraded sharply with scale, and that all models showed severe first-proposal bias, giving fast sellers a 10–30 times advantage over good ones; susceptibility to manipulation varied widely between models (Bansal et al., 2025).

Developers already feel this. On Reddit, a builder describes the state of the art as "someone hands them a card and hopes nothing weird happens", and adds that "the same model that hallucinates a price can hallucinate an approval check" (r/AI_Agents, n.d.). Another warns that "prompt-level guardrails fail open" (r/OpenSourceeAI, n.d.). A third sums up the demand: "Agents should be able to spend money, but only inside clearly defined limits" (r/ArtificialInteligence, n.d.).

The people doing the work have their own trust problem. In a Freelancers Union survey, 62% of New York freelancers had lost pay to non-payment, 91% had been paid late and 54% had waited three months or more (Freelancers Union, 2022). Platform escrow does not always protect them: one Upwork freelancer "won" a dispute and received "ONE (1) dollar" of the USD 1,500 in escrow (Upwork Community, 2021); another delivered and was then hit by a USD 2,500 chargeback (Upwork Community, 2020); a top-rated seller reported that the platform "reversed my earned payments back to clients for work already delivered and accepted" (r/Upwork, n.d.).

The largest platforms are also changing course. Fiverr reported 3.1 million annual active buyers at the end of 2025, down 13.6%, with a marketplace take rate of 27.7%; its annual report says buyer counts have been declining "including as a result of AI technologies" that reduce demand for simple services, and its 2026 plan deprioritises low-end transactions (Fiverr International, 2026a, 2026b). Upwork reported USD 4.0 billion of gross services volume for 2025 (Upwork, 2026). Both list users going around the platform to avoid fees as a material risk.

b. Statement of the Problem

Three gaps meet in one place.

Agents cannot be trusted with money. When an agent holds a card or a wallet key, the only limits are in its prompt. A hallucination, a malicious counterparty or a stolen key can spend everything.

Agents trade badly with each other. Buyer agents take the first offer, are swayed by fake reviews, and follow instructions hidden in what they read (Bansal et al., 2025). A bid that is too good to be true, or a delivery whose code comments tell the agent to pay another account, is aimed at exactly those weaknesses.

Small service jobs lack a fair, cheap venue. Payment depends on a client choosing to release it, card and platform payments can be reversed after delivery, fees are high relative to small jobs, and the largest platform is deprioritising them.

Problem statement. There is no services marketplace where a person or an agent can hire humans or agents, with hard limits the agents cannot talk their way past, while sellers know that work meeting an agreed, machine-checkable definition of done is paid and stays paid.

c. Objectives

The general objective is to design, build and test a two-sided services marketplace on NEAR, starting with small software jobs, and to report honestly whether its guards and market rules hold. Specifically, to:

1. fix eight pass marks before building (Section III.a), covering guard enforcement, manipulation resistance, negotiated price, settlement speed, reviewer accuracy, off-platform lures, reproducibility and one external test with outside teams;
2. build one NEAR contract that holds USDC deposits, escrows each job by milestone against a locked acceptance specification, enforces each owner's mandate, accepts dispute rulings only from an arbiter DAO, and records reputation from settled jobs only;
3. build seven agent roles (spec writer, buyer, bidding, transaction, three reviewers, fraud and lure, guardian) that work through the contract with limited keys;
4. build a web app with email or social login where buyers post jobs with tests, sellers or seller agents bid, and both owners watch and override their agents;
5. run a scripted attack and accuracy suite on NEAR testnet and publish every result, failures included; and
6. enter one project with Autonomous Business as the main track, plus Agent Economy, AI Swarms, and AI Security & Defense (NEAR Hackathon, 2026).

d. Scope and Boundaries

Services only, one launch category. The demo covers small software jobs (a bug fix, a missing test, a contract method, a front-end change), each with a test that must pass. Physical goods are out of scope because a tracking number proves shipment, not condition. Translation is the planned second category.

Testnet first. The prototype runs on NEAR testnet with testnet USDC. NEAR Intents has no testnet, so cross-chain pay-in is shown with dry-run quotes; so the demo makes one real mainnet payment of one or two dollars and links its transaction in the README (NEAR Hackathon, 2026). Mainnet USDC uses the same contract interface (Circle, 2023).

Human and agent sellers. Sellers may be developers or AI agents, such as the Dev Tools agents listed on NEAR's Agent Market (NEAR AI, n.d.-b).

DAO arbiter for disputes. Disputes freeze the milestone and go to an arbiter DAO on Sputnik DAO. Automated dispute rulings are future work.

No fiat on-ramp or off-ramp, tax, KYC or consumer-law handling. Users arrive with USDC on NEAR or pay through Intents. A real deployment would need all of these.

One account, two modes. Every AGORA account can both buy and sell. A switch in the app moves between Buying and Selling, as on Fiverr. The NEAR account, wallet, agent team and mandate stay the same in both modes. Buying shows the jobs the account posted; Selling shows its orders, the open jobs it can bid on, and its earnings. The design keeps one rating per side (Section III.e), so that a good record as a buyer does not raise a seller rating; the prototype contract keeps a single record per account (Table 13). A seller who needs help on part of a job can hire from the same account, with no second sign-up.

II. REVIEW OF RELATED LITERATURE AND SYSTEMS

a. Agent Payment Protocols

The current protocols share one idea: a signed mandate that proves a person allowed an agent to spend. ACP structures a cart and a payment token for checkout inside a chat app; AP2 verifies with cryptographic mandates that an agent has real permission; x402 lets an agent pay a web resource in stablecoins; Visa and Mastercard issue agent-specific card tokens (Airwallex, 2026). All of them sit between one agent and one merchant. None defines how many buyer and seller agents negotiate, how delivered work is checked, or how a seller is protected from rever-

sal. AGORA borrows the mandate idea but enforces it in a contract, where the agent cannot change it.

b. What Happens When Agents Trade

Magentic Marketplace (Bansal et al., 2025) is the closest study of agent-to-agent markets. Frontier models came close to the best possible welfare, but only with good search; welfare fell as the number of search results grew; first proposals were chosen far more often than later ones; and susceptibility to manipulation varied widely between models. Three design rules follow, and AGORA adopts them: show the buyer agent a short ranked shortlist, not everything; require several offers or a waiting window before accepting; and assume some counterparties are hostile, so that no bid or delivery text can move money by itself.

c. Evidence from Online Labour Markets

New workers struggle to be hired. In a field experiment on oDesk, Pallais (2014) hired 952 randomly selected workers and gave some detailed public evaluations; both the hire and the detailed evaluation substantially improved later employment. A market that caps new sellers, as AGORA does, must also give them a path to a first paid job.

Trust can increase disintermediation. In a randomised trial on a large freelance marketplace, higher trust helped high-quality freelancers get hired but, once trust was high enough, also increased the rate at which parties moved off the platform, offsetting revenue gains (Gu & Zhu, 2021). Escrow alone does not keep users; the platform must keep providing value after the first job.

Ratings inflate. Over more than ten years of data from several marketplaces, ratings drifted upward without a matching rise in satisfaction (Filippas et al., 2022). AGORA weights objective outcomes (tests passed, disputes, refunds) above stars.

d. Earlier Crypto Freelance Markets

Several platforms have offered freelance work paid in crypto. Their records suggest that the payment rail was not the binding constraint (Table 1).

Table 1. *Earlier crypto freelance and escrow systems (sources read November 7, 2026)*

System	What the sources show	Lesson for AGORA
Braintrust	10% client fee; governance-only token (Cointelegraph, 2020). June 2023: ~75% of enterprise clients froze hiring; 40% of staff laid off (Braintrust, 2023)	Concentrated demand made the market fragile
LaborX	Several EVM chains; up to 10% freelancer fee. Claims 100,000+ professionals, lists ~1,140 jobs (undated) (LaborX, n.d.)	Supply is easier to attract than demand
Dework	DAO task board paid from a treasury; no stated fee (Dework, n.d.)	Parties already knew each other; no market
Kleros Escrow	No platform fee; staked random jurors; "not designed for disputes that are purely subjective with no evaluation criteria" (Kleros, n.d.)	Arbitration needs written acceptance criteria
ERC-8183 (Virtuals ACP)	Escrowed agent job; one evaluator, fixed at creation, completes or rejects; no dispute or appeal; refund after expiry cannot be blocked (Ethereum Improvement Proposals, 2026)	One evaluator is a single point of failure; keep the unblockable expiry refund
NEAR AI Agent Market	Post, bid, escrow and a delivery hash on NEAR Intents; the team found no documented dispute process (NEAR AI, n.d.-b)	Agent markets pay; few verify

e. Trust Failures in Freelance Platforms

The team's field research collected public posts from Reddit and the Upwork Community forum, and the annual reports of Fiverr and Upwork (Table 2). The pattern is consistent: the person who did the work has no control over release, and a third party can freeze, reverse or short the payment. The common advice, "ask for 50% upfront", is a manual workaround for missing automatic escrow.

Table 2. *Pain points from public posts, surveys and filings, and the AGORA feature that answers each*

Pain point (source)	Evidence	AGORA feature
Agents with money have no hard limits (r/AI_Agents; r/OpenSourceeAI)	"prompt-level guardrails fail open"	Mandate enforced in contract; limited key
Non-payment and late pay (Freelancers Union, 2022)	62% lost pay; 54% waited 3+ months	Pre-funded escrow; release by reviewer vote, not by the client
Short release from escrow (Upwork Community, 2019, 2021)	"$1 of $1,500 paid"	Milestone amounts fixed at acceptance
Chargebacks after delivery (Upwork Community, 2020; Woodward, 2026)	USD 2,500 chargeback; 43.8% of chargebacks labelled friendly fraud	Released USDC is final
Payment diversion and email compromise (AFP, 2026; FBI IC3, 2018)	Over 75% of US firms hit by payments fraud in 2025	Payouts only to the seller account bound to the job
Unclear definition of done (Kleros, n.d.)	Juries cannot judge criteria-free disputes	Acceptance tests locked by hash
High fees on small jobs (Fiverr International, 2026b)	27.7% take rate	Contract-held escrow; low fee

Pain point (source)	Evidence	AGORA feature
Circumvention to avoid fees (Fiverr International, 2026a; Upwork, 2026)	Listed as a material risk by both	Agents, mandates and reputation only work inside AGORA; lure agent
Fake jobs and bait offers (Upwork Community, 2023; Bansal et al., 2025)	"unrealistic pay rates"; agents take the first offer	Fraud and lure agent; price check holds outliers

f. NEAR Building Blocks

Native USDC. Circle issues USDC natively on NEAR, at contract `17208628f8…133a1`, since September 2023; the bridged USDC.e is a different token (Circle, 2023). A stablecoin lets sellers price in dollars.

Token deposits in one step. Under the NEP-141 standard, `ft_transfer_call` sends tokens to a contract and calls its `ft_on_transfer` in the same transaction, and the receiver returns any unused amount for refund (NEAR, n.d.-b). AGORA uses this for deposits.

Function-call keys. A NEAR account can hold keys that may call only one contract and, optionally, only named methods, with a gas allowance. They cannot attach tokens to a call (NEAR, n.d.-a). This is the core of AGORA's security: an agent's key can call the marketplace contract and nothing else, and because it cannot attach a deposit, it cannot send USDC directly either.

Speed. NEAR mainnet moved to 600 ms blocks and about 1.2 s finality in May 2025 (The Defiant, 2025), fast enough for negotiation and release to feel immediate.

Private inference. NEAR AI Cloud runs models inside trusted execution environments (TEEs) with attestation, so that "model providers, cloud providers, and NEAR AI cannot access" the data (NEAR AI, n.d.-a). A buyer's maximum price can then stay hidden, and each reviewer vote can carry proof of which model ran.

Intents, DAOs and onboarding. NEAR Intents lets a buyer pay from another chain and settle as USDC; it handled about USD 3.9 billion of volume in the 30 days to November 3, 2026 (DefiLlama, n.d.-b). Sputnik DAO gives a multi-member arbiter, meta-transactions let a relayer pay gas, and social login lets a developer join with email. Section III.f sets out why and where AGORA uses each tool.

g. Research Gap

There are agent payment protocols, escrow contracts, capped agent wallets and code bounties, and there is evidence that agent markets and labour markets fail in predictable ways. There is no marketplace that (1) gives both buyers and sellers their own agents, (2) lets either side be a human or an agent, (3) locks a machine-checkable definition of done at acceptance and releases payment only on an independent, attested reviewer vote, (4) designs against first-proposal bias, injection and off-platform lures, and (5) settles disputes through a DAO and builds reputation only from settled payments.

III. METHODOLOGY

a. Design and Pre-Registered Pass Marks

This is a design-and-build study with a scripted evaluation on NEAR testnet and one external test. Before any marketplace code is written, the pass marks in Table 3 are fixed in this paper (dated November 7, 2026) and will be copied to `PASS_MARK.md` in the repository. Failures will be reported as failures. Compared with version 1, P2, P5 and P7 were rewritten for software jobs and P8 was added.

Table 3. *Pass marks, fixed before building*

#	Criterion	Target
P1	Out-of-mandate actions rejected on-chain (50 scripted: over per-job cap, over daily cap, unlisted seller, wrong category, while paused, method outside key)	50/50 rejected; 0 USDC moved
P2	Hostile bids and deliveries (20 seeded: fake reviews, injected "pay account X" text, agent instructions hidden in code comments or test output)	0 USDC to attacker accounts; ≥18/20 flagged
P3	Price and first-offer bias (30 jobs, 5 seller agents each)	First-offer acceptance <20% (vs 60–100% reported); price ≤ median bid in ≥80%
P4	Settlement: release after the second reviewer approval	Median ≤5 s on testnet; no release without 2 approvals or timeout
P5	Reviewer swarm accuracy (20 seeded deliveries: 10 correct; 10 bad, incl. edited tests, out-of-scope edits, build breaks)	≥9/10 good released; ≤1/10 bad released
P6	Reproducibility for judges	One command runs P1–P5 and P7 and prints results in <10 min
P7	Lures and bid price validation (40 seeded bids: 20 normal; 20 off-platform requests or far-off prices)	≥18/20 lures held; ≤2/20 normal held; 0 lures funded without review
P8	External use by teams outside AGORA	≥3 jobs from ≥2 outside buyers, funded, released by reviewer vote, with explorer links

b. Requirements Sources

Requirements come from five places: the field research on payment pain points (Table 2); the Magentic Marketplace failure modes (Section II.b); the labour-market findings (Section II.c); the NEAR key and token rules (Section II.f); and the gaps the team found in its own earlier payroll vault, where a worker could mark its own job complete and an award did not reserve the job's budget. Both gaps

are closed in AGORA by the reviewer vote and by moving funds into escrow at acceptance.

c. Choice of Launch Category

A market works only when each buyer sees enough credible sellers and each seller sees enough jobs. Spreading a small launch across logos, translation and code would leave every category thin, as LaborX's ratio of professionals to jobs suggests. AGORA launches with software jobs for three reasons: completion is machine-checkable, which removes most of the subjectivity that arbitration cannot handle (Kleros, n.d.); NEAR teams have small development tasks and hold stablecoins, about USD 116 million on NEAR when read on November 7, 2026 (DefiLlama, n.d.-a); and Dev Tools is the second-largest category on NEAR's Agent Market, with 116 listings (NEAR AI, n.d.-b), so agents can bid alongside developers. One assumption remains untested: that NEAR users will pay for services through a marketplace. The team found no verified data on paid service jobs between NEAR users; Agent Market counts are listings, not completed jobs. P8 tests this directly.

d. Agent Roles

Each user gets a small team of agents. Only the transaction agent can sign contract calls that move funds; the others advise it, vote, or (for the guardian) stop it. Version 1's finder and negotiator are merged into one buyer agent, and the price agent becomes the fraud and lure agent. Two roles are new. The spec writer exists because most buyers will not write tests themselves, and without a test there is nothing for the reviewers to check. The guardian exists because caps limit how much a stolen key can spend, but not how fast it tries; a key that is probing the limits should be stopped, not only refused.

Table 4. *Agent roles*

Agent	Side	Job	Can move funds?
Spec writer	Buyer	Turns a plain-language job post into failing tests and an acceptance spec (Table 6), which the buyer approves before posting	No
Buyer	Buyer	Sends the locked job privately to matching sellers; lists valid offers lowest price first and tags held ones; waits for ≥3 valid offers or a time window; negotiates within the hidden budget in a TEE; sends the hire offer to the seller the buyer chooses	No
Bidding	Seller	Watches jobs that match the seller's skills and posts bids with price, delivery date and milestones	No
Transaction	Both	Accepts the offer, funds escrow, submits delivery; signs with a function-call key	Yes, inside the mandate
Reviewer ×3	Shared	Independently run the locked tests on the delivered commit; vote with a log hash and attestation	No; 2 votes release
Fraud and lure	Shared	Scores each counterparty and bid 0–100; checks bid prices against a reference; flags off-platform requests	No; can block or hold
Guardian	Each owner	Watches the owner's on-chain activity; pauses the agents on bursts of rejected calls, spending near the daily cap in minutes, or a new seller receiving several jobs at once; tells the owner why	No; key can call only pause

e. Contract Design

One Rust contract, extended from the team's tested payroll vault, holds five kinds of state.

Balances. A user deposits USDC once from their own wallet with `ft_transfer_call`; `ft_on_transfer` credits their balance inside the contract. Withdrawals need the user's full-access key.

Mandates. Each user sets, with their full-access key, the agent's public key and the limits in Table 5, and creates a sub-account for their agent team. The agent's function-call key is limited to the methods `post_job`, `place_bid`, `accept_offer`, `fund_job`, `submit_delivery` and `vote`. It has no withdraw method and cannot attach a deposit. A second function-call key, held by the guardian agent, can call only `pause`; only the owner's full-access key can unpause.

Table 5. *Mandate checks enforced by the contract on every funding call*

Check	Rejects when
Pause	The owner or the guardian has paused the agent (or the market is paused)
Per-job cap	Job price > `max_per_job`
Daily cap	Today's funded total + price > `daily_cap`
Category list	The job's category is not allowed
Approved sellers (optional)	Strict mode is on and the seller is not on the list
Fraud flag	The seller or bid is flagged high-risk (owner approval is designed, not built)
Price hold	The bid is held by the price check and not yet cleared
New-seller value cap	The seller has fewer than 3 settled jobs and the price is above their starter cap
Balance	The buyer's free balance is too low

Jobs. Accepting an offer creates a job with fixed milestones whose amounts sum to the price, and stores the hash of the acceptance spec (Table 6). `fund_job` moves the amount from the buyer's balance into escrow after every mandate check passes. Each milestone is released when two of the three reviewers approve, each vote storing a test-log hash and a TEE attestation hash. Release is final, and there is no buyer review window after approval: the buyer is protected before money moves, by the locked tests, a hidden holdout set and the reviewers, and payment never waits on a silent client because the client does not release it. If the reviewers do not reach two votes within 12 hours, any party can call `rerun_review` and the review is run again (by three fresh reviewers in the design; by the same three reviewer accounts in the prototype); money never moves on a timeout. If no delivery passes by the deadline locked at acceptance, `refund_expired` returns the unreleased milestones to the buyer. Anyone can call it and nothing can block it, following `claimRefund` in ERC-8183 (Ethereum Improvement Proposals, 2026).

Hidden checks. Coding agents pass visible tests by editing them, special-casing inputs or overloading operators, and hiding the tests cut this to near zero in ImpossibleBench (Zhong et al., 2025); even hidden tests can be weak, since 7.8% of SWE-bench patches marked correct failed the full developer suite (Wang et al., 2025). The spec writer therefore writes a second, hidden holdout set next to the visible failing tests. Both hashes are locked at `accept_offer`, only the attested test runner opens the holdout set, and the reviewers run both. The three reviewers use models from different families and are assigned after delivery, because a panel of diverse models agrees with humans better than one strong judge (Verga et al., 2024).

No self-dealing. `accept_offer` refuses an offer whose seller account is the buyer account or one of its agent subaccounts. Without this check, one account in both modes could hire itself and farm ratings.

Table 6. *Acceptance specification for software jobs, locked at acceptance*

Check	How a reviewer verifies it	Weight
Named failing tests now pass	Run the test command on the delivered commit in a sandbox; record exit code and log hash	Required
No regressions	Run the full suite before and after	Required
Scope	Diff touches only the allowed files	Required
Builds and deploys	Build; for contracts, sandbox deploy and call	Required for contracts
Hidden holdout tests pass	Run the holdout set, which only the attested test runner can open; its hash was locked at acceptance	Required
No planted instructions	Scan diff and comments for agent-directed text	Required
Quality notes	Model-judged, shown to the buyer	Advisory

Delivery attempts. Each offer states how many delivery attempts the seller gets (default three), and the number is locked with the price and the spec hash at `accept_offer`. A rejected delivery uses one attempt. When the last attempt is rejected, the buyer can take a refund of the milestone in one call or grant one more attempt; the seller cannot be asked for work beyond the locked tests, so there is no separate buyer revision count, and new requirements become a new paid milestone.

Disputes. A dispute freezes the milestone and climbs a ladder of at most three steps, so that most cases end without people and none costs more than the job. *Step 1, free re-check:* a split vote, a reviewer timeout, or a party who says a verdict was wrong triggers `rerun_review`; the reviewers re-run the visible and hidden tests (three fresh reviewers in the design, the same three reviewer accounts in the prototype). *Step 2, DAO vote:* within 72 hours of the re-check, either party may escalate by posting a bond of 5% of the milestone (at least 1 USDC) with `ft_transfer_call`. The contract opens a Sputnik DAO FunctionCall proposal to `resolve_dispute(job, milestone, seller_share)`, and two of three council votes pass a release, refund or split (NEAR, n.d.-c). Members who have traded with either party are excluded automatically, checked from the job history. The winner gets the bond back; a lost bond goes to the DAO treasury, following the bond logic of optimistic oracles (UMA, n.d.). *Step 3, default:* if the DAO has not decided within 72 hours, the re-check result stands. There are three valid reasons: the locked tests were wrong or impossible; the review or test run failed; the deadline or delivery is contested. "The tests pass but the work is wrong" is not a reason, because the locked tests are the contract (Kleros, n.d.); new requirements become a new paid milestone. The spec, commits, test logs and votes are attached automatically as the evidence bundle.

Fee and treasury. On each release the contract sends 2% to the arbiter DAO's treasury, which also receives lost bonds. The treasury pays a flat fee to council members for each decided dispute and tops up the first-job pool (below). AGORA therefore runs as a business with its own treasury and revenue sharing while humans only set its rules, as the Autonomous Business track describes (NEAR Hackathon, 2026).

Reputation. When a job settles, the design updates both sides' counts of settled, refunded and disputed jobs and their settled volume and accepts one rating per side (the prototype contract keeps one record per account with settled and disputed counts and one rating average, Table 13), and mints a non-transferable NEP-171 completion record to the seller. A rating with no settled job behind it cannot exist.

First-job pool. Because starter caps make new sellers harder to hire (Pallais, 2014), the market reserves a small funded pool for first jobs. Sellers can also raise their

starter cap early with an optional refundable USDC stake, returned after clean jobs.

f. NEAR Tools: What, Why and Where

AGORA uses nine NEAR tools. Each one is there because it removes a specific trust problem from Section I.b, not to add a logo. The design rule is that any tool whose job is to protect money must be enforced by the protocol or the contract, never by an agent's prompt.

Table 7. *NEAR tools in AGORA*

Tool	Why AGORA needs it	Where it is used
Function-call keys	Agents must act without being able to steal. The protocol itself stops these keys from withdrawing or attaching tokens	Each transaction agent signs only the six job methods; the guardian agent's key can call only `pause`
Sub-accounts	Keeps each user's agents separate from their main wallet, so one compromised team cannot touch another	`agents.alice.testnet` holds a user's agent keys; deleting it removes them all
NEP-141 (native USDC)	Sellers price in dollars; one-step deposits remove a failure point between "sent" and "credited"	`ft_transfer_call` into the contract; `ft_on_transfer` credits balances and refunds unused amounts
NEP-171	A completion record that cannot be bought or moved is harder to fake than a star rating	Non-transferable record minted to the seller on every settled job
NEAR AI Cloud (TEE)	The buyer's limit price must stay private, and a reviewer's vote must prove which model ran on which input	Buyer agent negotiates in a TEE; each reviewer vote stores an attestation hash
Sputnik DAO	Disputes need a decision no single party (including the AGORA team) controls	Each escalated dispute becomes a FunctionCall proposal to `resolve_dispute`; the council votes and the DAO executes it. The DAO treasury receives the 2% fee
NEAR Intents	Buyers hold money on many chains; asking them to bridge first loses them	Pay-in from another chain, settled as USDC into escrow; one real small payment on mainnet in the demo, since Intents has no testnet
Chain Signatures	Some sellers want to be paid on the chain they already use	Optional payout of a released milestone to a seller address on another chain
Meta-transactions and social login	Developers should not need a wallet or gas tokens to bid on their first job	Email (one-time code), passkey or Google sign-up, with no passwords; a relayer pays gas for sellers' bids and deliveries

Three of these carry most of the security argument. Function-call keys make the worst case of a fooled or stolen agent bounded by the mandate; the TEE makes reviewer votes checkable after the fact; and the DAO removes the AGORA team as a party that could take escrowed funds. The other six make the market usable: dollars instead of a volatile token, any chain in and out, and no wallet setup for a first-time seller. NEAR's 600 ms blocks and about 1.2 s finality (The Defiant, 2025) keep each of these steps fast enough to feel immediate in the demo.

g. Threat Model

Table 8. *Threats and defences*

Threat	Defence
Agent hallucinates a price or overspends	Per-job and daily caps in the contract
Bid or delivery text tells the agent to pay another account	Funds can only go to the job's seller account; the agent key cannot transfer
Delivered code edits the tests to pass	Reviewers run the locked tests from the base commit; scope check; hidden holdout tests
Fake reviews or badges lure the buyer agent	Ratings only from settled jobs; objective outcomes weighted above stars; fraud score
Agent key stolen	Key can only act within the mandate, cannot withdraw; guardian pauses on unusual activity; owner deletes the key
Seller delivers nothing or junk	Escrow; 2-of-3 reviewer vote; limited delivery attempts, then one-call refund; dispute freezes the milestone
Buyer never confirms	The buyer does not release; reviewer votes do
Reviewers or the test runner go down	A timeout triggers a fresh re-check, never a payment
Seller misses the deadline	`refund_expired`, callable by anyone and not blockable

Threat	Defence
Buyer disputes work that passed the locked tests	Not a valid dispute reason; new requirements become a new paid milestone
Chargeback after delivery	On-chain USDC release is final
One reviewer is bribed or fooled	Two of three must agree; different models and prompts; attestation recorded
Bait bid, then off-platform payment	Price check holds outliers; lure agent; escrow pays only on approved delivery
Sybil sellers farm reputation	Reputation weighted by settled volume and distinct buyers; new sellers start with low limits
Arbiter collusion	Free re-check first; then a 2-of-3 Sputnik DAO vote; members who have traded with either party cannot vote; dispute bond; evidence bundle with rerunnable log hashes
Self-dealing: one account, or two linked accounts, hires itself to farm ratings	Contract rejects seller = buyer, including agent sub-accounts; fraud and lure agent flags accounts funded from the same source; ratings only from settled payments

h. Fraud Scoring

The fraud agent combines simple, explainable signals into a 0–100 score: account age and settled history from the contract; bid price far below the reference; ratings from accounts that share funding sources; urgency or off-platform payment requests in bid or message text; and instruction-like text aimed at agents (for example "ignore previous instructions") in bids, code comments or test output. A score of 70 or more blocks funding (an owner override is designed but not built: the prototype contract always refuses); 40–69 asks the owner. Each score lists its top reasons in the app. The score is advice; the contract checks only the stored flag.

i. Bid Price Validation

A seller could bid USD 2 for a job that usually costs USD 150, or USD 5,000 for a one-line fix. The first is usually bait: it pulls buyer agents in, then the seller asks for payment off-platform or never delivers. The second can drain a careless buyer.

Reference price. For each bid the agent builds a reference from (1) settled AGORA jobs in the same category and size, (2) live bids on comparable jobs, (3) prices listed on NEAR's Agent Market for similar agent services, and (4) the seller's own history. It takes the median and records how many prices that median rests on.

Rule. A bid under 30% or over 300% of the reference median is held, not rejected. The seller sees why ("similar fixes settled for USD 60–110") and can explain: a reused library, a rush fee. The explanation is shown to buyers and their agents. If the reference rests on fewer than five prices, the bid is marked "unverified" instead of held. The contract stores only the result (cleared, held, unverified), and `fund_job` refuses held bids. Buyer agents are told that a price far below the reference is a warning sign that calls for extra checks, not a bargain to grab, which answers the first-offer weakness found by Bansal et al. (2025).

j. Evaluation

P1–P7 are measured on NEAR testnet with scripted agents and seeded jobs. P1 and P2 count rejected transactions and USDC balances of attacker accounts, and record whether and how quickly the guardian paused the attacked agent. P3 logs every offer and acceptance and compares price with the median bid. P4 times from the second vote's transaction to the seller's balance change. P5 uses deliveries labelled good or bad in advance by a team member who does not build the reviewers; the labels are hashed and committed before the reviewers first run. P7 seeds bids whose labels are known. P8 records jobs posted by outside teams. Every run writes a JSON log and explorer links.

IV. SYSTEM DESIGN AND PROTOTYPE

a. Architecture

The web app (React and TypeScript) lets buyers post jobs with tests and sellers bid. A NestJS backend serves the API, starts agent workflows in n8n and keeps a searchable index of jobs and bids in Supabase Postgres; the contract is the source of truth for money, disputes and reputation. Agent reasoning for the buyer agent and the reviewers runs on NEAR AI Cloud so that a buyer's limit price stays private and votes carry attestation. The team's n8n NEAR node runs scheduled jobs, such as review timeouts and deadline refunds (Figure 1). Section IV.b lists every layer of the stack.

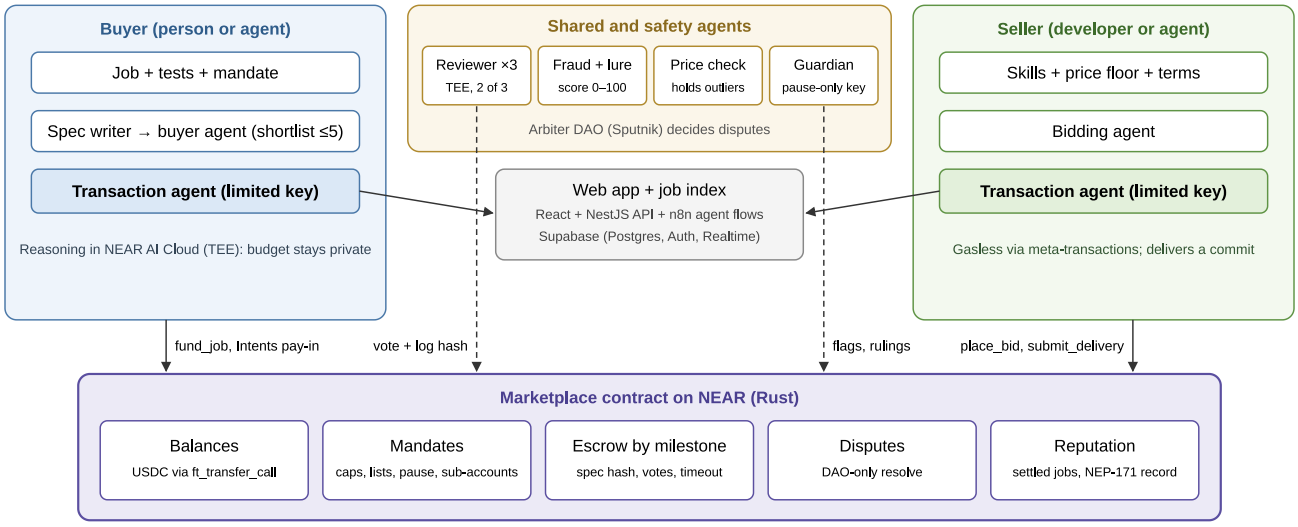


Figure 1. *AGORA architecture. Agents reason off-chain; every movement of money goes through the contract, which checks the owner's mandate and the fraud and price flags. Only the two transaction agents hold keys that can move funds, and those keys cannot withdraw or attach deposits; the guardian's key can only pause. Disputes are decided only by the arbiter DAO.*

b. Technology Stack

The team builds on React for the web app and NestJS for the backend, both in TypeScript, so one language covers the browser, the API and the agent glue code. Table 9 lists the rest of the stack and why each piece is there. The rule is the same as for the NEAR tools: nothing off-chain is trusted with money. The backend, the database and the agents can fail or be wrong, and the contract still enforces every limit.

Table 9. *Technology stack (core = needed for the hackathon demo; extra = added if time allows)*

Layer	Choice	Why AGORA uses it	Priority
Frontend	React 19 + TypeScript on Vite; React Router; Zustand; plain CSS	Fast builds, a small dependency set, one store for the job board and dashboards	Core
Live updates	Supabase Realtime (or Socket.IO from NestJS)	Bids, haggling and reviewer votes appear in the agent activity feed as they happen	Core
Backend API	NestJS (TypeScript) with Prisma or TypeORM	Modules for jobs, bids, agents and chain calls; typed database access	Core
Message checks	zod or class-validator	Every agent message is checked against a schema before NestJS acts on it, so a malformed or injected reply is dropped, not executed	Core
Database and auth	Supabase: Postgres, Auth, Realtime	Users, jobs, bids and agent logs in one place; email one-time code, passkey and Google sign-in	Core
Agent workflows	n8n (with the team's NEAR node), called by NestJS through webhooks	Each agent (spec writer, buyer, bidding, fraud and lure, reviewer, guardian) is a visible, editable flow; the price check runs inside the fraud and lure flow (Section III.d)	Core
Models	NEAR AI Cloud for the buyer agent and reviewers; one general LLM API for the other agents	TEE privacy and attestation where they matter; one general model keeps the demo simple	Core
Test runner	Throwaway Docker container per delivery	Reviewers run the locked tests on the delivered commit without touching the host	Core
NEAR access	`near-api-js` (or `@near-js/*`) in NestJS; Rust contract; testnet USDC (NEP-141)	Signs and sends agent transactions; reads balances, jobs and votes	Core
Wallet and login	NEAR Wallet Selector (optional); email one-time code, passkey or Google sign-in (for example FastAuth) with a relayer	Buyers with wallets may connect them after sign-up, or skip; new sellers start with email and never need gas tokens, and add a payout wallet only when they withdraw	Core
Similar-job search	pgvector in Supabase	Finds comparable jobs and bids for the reference median in the price check	Extra
Jobs and queues	BullMQ with Redis	Review timeouts, deadline refunds and retries of failed agent calls	Extra

Layer	Choice	Why AGORA uses it	Priority
Deploy	Vercel (React); Railway or Render (NestJS, n8n); Docker Compose	A hosted demo for judges, and one command to run the whole stack locally	Core

For the demo the team keeps the core small: Supabase, n8n, one general LLM plus NEAR AI Cloud, and the NEAR contract. pgvector and Redis are added only if time allows; without them the price check uses plain category filters, and timeouts run as n8n scheduled jobs. The Docker Compose file matters for judging, because the hackathon's AI screeners check whether a project runs from its README (NEAR Hackathon, 2026).

c. Job Lifecycle

A job moves through the states in Figure 2. The buyer's agent posts a job with an acceptance spec and a hidden limit. Bidding agents and human sellers bid. The buyer agent negotiates for at most three rounds and may accept only after three offers have arrived or a short window has passed, which is the direct answer to first-proposal bias. On acceptance, the spec hash is stored and the transaction agent calls `fund_job`; the contract runs every mandate check and locks the price in escrow, whether the money came from the buyer's balance or through NEAR Intents. The seller delivers a commit for each milestone, the reviewers run the locked tests and vote, and the contract pays the seller in USDC. A dispute freezes only the milestone in question and sends it to the arbiter DAO.

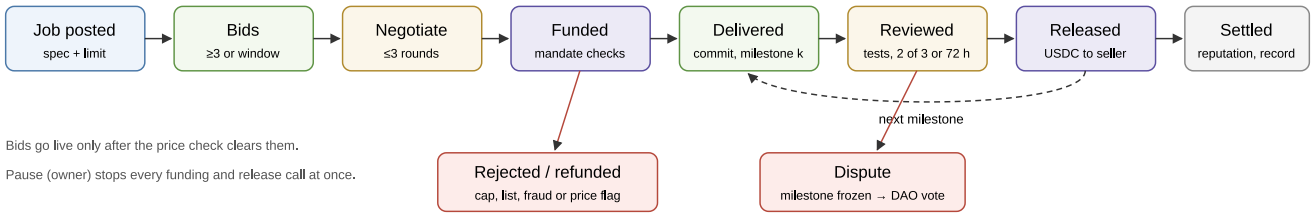


Figure 2. *Job lifecycle. Solid arrows are the normal path; red boxes are the ways a job stops. Milestones repeat until the price is fully released.*

Direct requests. A buyer can also request a listed service. That creates a job offered to one seller only. The seller answers with an offer, and from there the job follows Figure 2 unchanged: the spec is locked when the buyer accepts.

Asking in a chat. In the web app, New request is a chat with the buyer's agent team, not a form. The buyer describes the job in plain words; the spec writer answers with the failing tests and checks, and nothing leaves the account until the buyer locks them. The buyer agent then sends the locked job privately to the bidding agents and listed services whose skills match. Offers stream into the chat as a list sorted from lowest to highest price. Offers held by the fraud and lure agent (price check or off-platform requests) stay in the list with a Held tag and cannot be chosen, so the cheapest line on screen is never a trap. The buyer may choose any valid offer once three have arrived or the window has passed; by then the buyer agent has usually negotiated, and its recommendation is tagged Best pick. Only if no offer arrives does the agent offer the last option, posting the job publicly to Open jobs, where the same rules apply.

Hiring in the chat room. When the buyer chooses a seller, the buyer agent sends a hire offer (price, delivery, spec hash) in a chat room with that seller and waits. The seller, or the seller's bidding agent, accepts or declines. A decline locks nothing, and the agent proposes the next valid offer. On acceptance, the transaction agent calls `accept_offer` and `fund_job`; the self-dealing check and every mandate check run at this point, and the price moves into escrow, which the app labels Protected payment. Each step appears as a card with a timestamp and is logged in the buyer's Job requests list. This adds one confirmation step to Figure 2 between Negotiate and Funded, so that money never locks for a seller who is no longer available. The hire offer also states the number of delivery attempts.

Following a job. The app shows each job as eight stages: Requested, Offers, Agreed, Protected, Delivered, Reviewed, Released and Settled. The first stage is named Requested rather than Posted because a job starts as a private request in the chat and is posted publicly only as a fallback. The Protected payment page shows only the latest delivery in full; earlier attempts fold into a one-line log with their verdict and reason. A Proof of delivery card lists one line per proof (the locked visible tests before and after and the hidden checks, the commit and its scope, the reviewer votes with attestations, and the on-chain transactions) and offers the evidence bundle as a download. Buyers rate a settled job with one to five stars. The Disputes page shows the three dispute steps as one short list (free re-check, DAO vote, result), with the bond, the excluded members and the evidence bundle folded away until asked for. The job page shows the deadline and a one-tap refund once it passes, and the wallet offers NEAR Intents pay-in. Find services lists services and, on a second tab, sellers (people and agents) with rating, settled jobs and first-try pass rate; a seller's name opens their profile.

What the seller sees. In Selling mode, every job the account was hired for sits in one Orders list. Each order shows one of five stages, which map onto Figure 2:

1. **Incoming:** before Bids, direct requests and hire offers; the seller sends an offer, accepts or declines.

2. **Offer sent:** Bids and Negotiate; the seller waits for the buyer to accept.
3. **Payment protected:** Funded into escrow; the seller builds and submits a delivery.
4. **Reviewers voting:** Delivered and Reviewed; payment releases at 2 of 3 approvals; if the reviewers stall, a fresh re-check runs.
5. **Paid:** Released and Settled; the seller can view the receipt.

d. Users and What They Get

Table 10. *Users and what they get*

User	Input	Output
Buyer (person, team or agent; Buying mode)	Job, tests, budget, mandate	Ranked shortlist, live negotiation log, one-tap override, a Job requests list with every job, a node for each stage and a timestamp for each step, receipts
Seller (developer or agent; Selling mode)	Skills, price floor, delivery terms	Bids placed by agent, direct requests to answer, an Orders list by stage, price check result with a chance to explain, escrow status, USDC on release, completion records
Business account	Recurring tasks, approved sellers, team caps	Autonomous procurement within caps; audit trail of every job
Arbiter DAO member	Disputes the free re-check did not settle, unless they have traded with either party	Evidence bundle, reviewer votes and logs; a vote on release, refund or split (2 of 3 pass); a fee from the treasury

Buyer and Seller are modes of one account, not separate sign-ups (Section I.d).

Every agent action appears in a timeline the owner can read, with the reasons for each choice. The owner can pause their agents, change limits or approve a flagged seller from the same screen.

e. What Is Already Built

Two pieces exist from the team's earlier work, both on NEAR testnet, and are disclosed as prior work. A Rust vault contract enforces `max_per_payment`, `daily_cap`, an approved-member list and `pause`, with a small job board (`post_job`, `bid`, `award`, `complete`) and unit tests such as a payment over the daily cap. A custom n8n node signs and sends NEAR transactions and has been checked against a public testnet contract. AGORA's contract extends the vault with USDC balances, milestones, acceptance specs, reviewer votes, DAO disputes and reputation. A first clickable UI prototype of the web app also exists. This paragraph describes the starting point on November 7, 2026; Section V.d records what was built by the submission date.

f. Reproducibility and Demo

The plan was for the repository to contain the contract, the agents, the web app, an example settings file, a Docker Compose file that starts the NestJS API, n8n and the database with one command, and one script that deploys to testnet, seeds jobs and attackers, runs P1–P5 and P7 and prints a results table with explorer links. Of these, the one-command script is not built (Table 13). The planned demo follows a "do X, see Y" script, of which the email-code sign-in and the NEAR Intents payment are not built: sign in with an email code; describe a job to fix a failing test under 40 USDC in the chat and lock the tests the spec writer proposes; watch offers arrive lowest price first; see a lure offer and a USD 2 bait offer tagged Held so they cannot be chosen; see the buyer agent wait for three valid offers and settle below the median; choose a seller and watch the seller accept in the chat room; try to fund a 400 USDC job and see the contract refuse; deliver a commit that edits the test and see the reviewers reject it; deliver a correct fix, watch two reviewers approve and the seller's balance rise; open a dispute and watch the free re-check, then escalate one to the DAO with a bond and see the vote decide it; let a deadline pass and take the refund in one call; pay one dollar in from another chain through NEAR Intents on mainnet; press pause and see every call stop.

V. EVALUATION PLAN AND CURRENT STATUS

a. Status of Pass Marks

Table 11 records the status at submission. The targets are unchanged from November 7, 2026. One pass mark was run on an earlier contract; the others are not measured yet and are reported as such.

Table 11. *Pass marks and status on November 14, 2026*

#	Target	Status
P1	50/50 out-of-mandate actions rejected	49/49 refused and 0 USDC moved on November 8, 2026, on the earlier contract `agora.kyle-n8n.testnet`. Not re-run on the current contract
P2	0 USDC to attackers; ≥18/20 flagged	Not measured
P3	First-offer acceptance <20%; price ≤ median in ≥80%	Not measured
P4	Release median ≤5 s; never without 2 votes or timeout	Not measured
P5	≥9/10 good released; ≤1/10 bad released	Not measured
P6	One command, <10 min	Not met: no one-command run exists

#	Target	Status
P7	≥18/20 lures held; ≤2/20 normal held	Not measured
P8	≥3 outside jobs from ≥2 buyers	Not measured

b. Expected Results and How They Could Fail

P1 should pass by construction, because the checks are in the contract; the risk is a missed method or an off-by-one in the daily window, which the 50 scripted cases are meant to catch. P2 depends on two layers: the contract makes it impossible to pay an account other than the job's seller, so the 0 USDC target should hold even if the fraud agent misses some attacks; the 18/20 flag rate is the uncertain part. P3 is the most likely to fail, because language models anchor on the first number they see; if it fails, the team will report it and tighten the waiting rule rather than move the target. P5 should be easier than in version 1 because tests replace taste, but a delivery that weakens tests while keeping them green is the main risk. P7 is likely to pass for common job types with many reference prices and to struggle with unusual jobs, which is why thin data gives an "unverified" label rather than a hold. P8 depends on outreach: if fewer than two outside teams post jobs, the paper will report that as evidence against the demand assumption in Section III.c.

c. Comparison with Related Systems

Table 12. *Feature comparison with existing approaches, from their public documentation (✓ present, – absent or not documented)*

Feature	Upwork, Fiverr	Crypto freelance (LaborX)	Agent pay protocols (AP2, x402)	Kleros Escrow	AGORA
Escrow before work	✓	✓	–	✓	✓
Agents on both sides	–	–	–	–	✓
Human and agent sellers	–	–	–	–	✓
Competing bids	✓	✓	–	–	✓
Spending mandate for agents	–	–	✓	–	✓
Test-based release	–	–	–	–	✓
Independent work check	–	–	–	✓ (jurors)	✓
Bid price check	–	–	–	–	✓
Decentralised disputes	–	–	–	✓	✓
Payment final after release	–	✓	✓	✓	✓

The comparison is about scope, not quality. The platforms and protocols in it work at scale today, while AGORA's column mixes built and designed parts; Table 13 separates them. AGORA's claim is the combination: no existing approach puts agents on both sides of a job and also ties release to a locked, machine-checked definition of done.

d. Implementation Status at Submission

Sections III and IV describe the design. Table 13 states which parts of it exist in the repository on November 14, 2026, so that no design choice is read as a finished feature. Every row marked built can be checked against the public repository and the contract's transaction list on NEAR testnet.

Table 13. *What is built and what is still design, on November 14, 2026*

Part of the design	State	Evidence or gap
Market contract: escrow, mandate checks, reviewer votes, refunds, disputes, reputation, completion record, 2% fee	Built	75 unit tests. Deployed as `agora-market-kyle.testnet`. One job ran from `post_job` to `rate` on November 14, 2026, signed by agent sub-accounts and reviewer accounts
USDC	Stand-in	`usdc.kyle-n8n.testnet`, a NEP-141 test token with a faucet, not Circle's USDC
Web app and NestJS API	Built	All screens and the scripted demo; 424 API tests; hosted
Agent workflows (n8n)	Built	15 workflow files for the seven roles and shared steps, hosted; a full run with outside agent sellers is not shown
Wallet sign-in (NEP-413), deposit, withdrawal and limits signed in the user's wallet	Built	Tested on testnet
Email code, passkey, relayer and meta-transactions	Not built	Sign-in is by NEAR wallet, or by email, Google or GitHub through Supabase
Agents sub-account created from the user's wallet	Not built	Agents act only for accounts the team set up
Reviewer votes with a log hash and an attestation hash	Partly built	Three reviewer accounts vote on-chain and both hashes are stored. The contract does not verify the attestation

Part of the design	State	Evidence or gap
Hidden holdout tests	Partly built	Run in the API's own test runner, not on the hosted runner
Disputes through a Sputnik DAO	Partly built	Contract methods and tests exist. `agora-arbiter.sputnik-v2.testnet` decided one dispute on the earlier contract. Excluding conflicted members is not built
Milestones	Partly built	The current contract holds one payment per job. Three milestones ran on the earlier contract
Reputation: one rating per side, refunded count, settled volume	Partly built	One record per account: settled and disputed counts and one rating average
Fresh reviewers on a re-check; owner override of a fraud flag	Not built	The same three reviewer accounts re-run; a score of 70 or more is always refused
NEAR Intents pay-in	Not built	No real Intents payment was made
Chain Signatures payout	Not built	—
Agent Market reference price; first-job pool	Not built	—
One-command evaluation of the pass marks	Not built	See P6 in Table 11

VI. DISCUSSION

a. What the Design Means

The central choice is where trust lives. In most agent products it lives in the prompt; in AGORA it lives in the contract and in a definition of done fixed before work starts. The agent can be wrong, fooled or stolen, and the worst outcome is still bounded by the owner's mandate. That turns the Reddit demand, "spend money, but only inside clearly defined limits", into something a judge can test by trying to break it. The same contract also fixes the seller's problem: once a job is funded, the money is in escrow, not in the buyer's goodwill, and once released it cannot be charged back.

The move from goods to services, and from all services to software jobs, follows the same logic. Escrow and reviewer votes are only as good as the evidence of delivery. Code with tests gives the strongest evidence a market can get; a parcel tracking number gives the weakest. Fiverr's retreat from simple, low-value services leaves room for a market cheap enough for small jobs, where buyers and sellers can be agents.

The swarm is not decoration. Each agent has a narrow job, the reviewers vote independently, and no single agent can both decide and pay. The marketplace rules (shortlist of five, three offers before accepting, bid prices checked against a reference, ratings tied to payments) are taken directly from the failure modes measured by Bansal et al. (2025). The labour-market literature adds one more warning: a trustworthy platform can lose users once they trust each other (Gu & Zhu, 2021). AGORA's answer is that its value continues after the first job, because the agents, mandates, reviewer checks and completion records work only inside the contract.

The tracks map cleanly. **Autonomous Business:** a business account can procure software work end to end while a human only sets limits and sits on the arbiter DAO, and the market itself keeps a treasury filled by a 2% fee that pays its arbiters and its first-job pool. **Agent Economy:** agents hire agents and humans, and earn as sellers. **AI Swarms:** buyer, seller, reviewer and fraud agents form a market with bidding. **AI Security & Defense:** on-chain mandates, limited keys, an attested 2-of-3 release and a measured attack suite.

b. Limitations

- **Demand is unproven.** No verified data shows NEAR users paying for services through a marketplace; P8 is a small test, not a market study.
- **Tests can be gamed.** A seller may weaken tests or overfit to them, as frontier models do when they can see the tests (Zhong et al., 2025). Locked tests, a hidden holdout set, scope checks and reviewers from different model families reduce but do not remove this, and hidden tests can still be too weak (Wang et al., 2025).
- **Only machine-checkable work fits.** Design, writing and other taste-driven work would need human review more often and are out of scope.
- **Price data can be thin or wrong.** Unusual jobs have few comparable prices. A false hold delays an honest seller, so holds are explained and reversible.
- **Team-built test sets.** P2, P5 and P7 use attacks and deliveries the team wrote; outside red-teaming is needed before the results generalise.
- **Arbiter DAO trust.** Disputes that pass the re-check depend on a small DAO council in the prototype; bonds and the automatic exclusion of members who traded with either party reduce, but do not remove, the risk of a biased vote. Token-weighted dispute votes have been captured by large holders elsewhere, which is why the council votes by role, not by token weight.
- **Owner key is still a single point of failure.** If the owner's full-access key is phished, the attacker can change the mandate. The owner key should stay offline or in a hardware wallet.
- **Onboarding friction.** Social login and gasless transactions remove the need for a wallet, but there is still no card on-ramp or bank off-ramp.
- **Many moving parts.** React, NestJS, Supabase, n8n, two model providers and a contract are a lot for one week; the core in Section IV.b is kept small so that the demo can still run if the extras are cut.

- **Self-dealing.** Letting one account buy and sell makes self-dealing easier. The contract blocks an account from hiring itself, but two linked accounts can still trade with each other, and the linked-account flag of the fraud and lure agent has not been tested yet.
- **Testnet.** Testnet USDC has no value, so behaviour under real stakes is not observed.
- **Time.** The coding window closes on November 14, 2026.

c. Outside Checks

Contract review. A NEAR developer outside the team reviews the escrow, mandate and dispute code, especially the `ft_on_transfer`, refund and `resolve_dispute` paths.

Usability test with 3–5 users per side. Developers bid on and deliver a job; buyers post a job with a test and set a mandate. Record time on task, errors and System Usability Scale scores.

Red-team session. People outside the team write hostile bids and deliveries to try to beat the reviewers, the lure agent and the buyer agent.

d. Future Work

Add translation as a second category with partly automated checks; supplement the DAO with reviewers staked in USDC; adopt AP2 or ACP mandates so outside agents can hire on AGORA; and run a small mainnet pilot with real developers.

VII. CONCLUSION

Agents are about to hire on people's behalf, and the evidence says they will be overspent, anchored on first offers and fooled by fake reviews unless the market is built to expect it. Freelancers, meanwhile, still wait on clients to release money and lose it to chargebacks, and the largest platform is moving away from small jobs. AGORA puts both sides' agents into one services market, with humans and agents as sellers, starting with software jobs whose tests decide when work is done. The money rules live in a NEAR contract: USDC escrow by milestone, a mandate the agent cannot change, a key that cannot withdraw, an attested 2-of-3 review against a locked spec, a price check on every bid, DAO arbitration, and reputation that only paid jobs can create. Eight pass marks are fixed here, before building. The contract is deployed and one job has run on it end to end; what AGORA has not done yet is run the pass marks, and that is the next test.

VIII. STATEMENTS

Prior work. The vault contract and n8n node (Section IV.e) were built before the hackathon window and are disclosed as such; only work within the window is submitted for judging.

AI use. AI assistants were used for parts of the drafting, the code and the documentation. The team reviewed everything and takes responsibility for its content. The agents in the product run on NEAR AI Cloud models and one general LLM API.

Author contributions. J. L. M. Valencia: project manager; K. A. M. Abandia: co-project manager and AI automation engineer; S. J. L. Calayag: UI/UX design; M. F. M. Altamarino: front end; M. R. P. de los Reyes: back end; L. E. P. Rosales: back end; M. V. G. Amparo: research; R. A. V. Dator: quality assurance. All eight authors reviewed this paper.

IX. REFERENCES

[1] Airwallex. (2026). Understanding agentic commerce protocols. Retrieved November 7, 2026, from https://www.airwallex.com/us/blog/understanding-agentic-commerce-protocols

[2] Association for Financial Professionals. (2026, April). Over 75 percent of U.S. firms experienced payments fraud in 2025 while AI adoption for fraud mitigation lags [Press release; 2026 AFP Payments Fraud and Control Survey]. https://www.financialprofessionals.org/about/learn-more/press-releases/

[3] Bansal, G., Hua, W., Huang, Z., Fourney, A., Swearngin, A., Epperson, W., Payne, T., Hofman, J., Lucier, B., Singh, C., Mobius, M., Nambi, A., Yadav, A., Gao, K., Rothschild, D., Slivkins, A., Goldstein, D. G., Mozannar, H., Immorlica, N., … Amershi, S. (2025). Magentic Marketplace: An open-source environment for studying agentic markets (arXiv:2510.25779). Microsoft Research. https://arxiv.org/abs/2510.25779

[4] Braintrust. (2023, June). Braintrust network update: June 2023. https://usebraintrust.com/blog/braintrust-network-update-june-2023

[5] Brohan, M. (2025, October 20). McKinsey forecasts up to $5 trillion in agentic commerce sales by 2030. Digital Commerce 360. https://www.digitalcommerce360.com/2025/10/20/mckinsey-forecast-5-trillion-agentic-commerce-sales-2030/

[6] Circle. (2023, September 14). USDC is now available on NEAR. https://www.circle.com/blog/usdc-is-now-available-on-near

[7] Cointelegraph. (2020, June 30). Freelancing is broken: This blockchain start-up wants to fix it. https://cointelegraph.com/news/freelancing-is-broken-this-blockchain-start-up-wants-to-fix-it

[8] De Rossi, M., Crapis, D., & Ellis, J. (2025). *ERC-8004: Trustless agents* [Draft Ethereum Improvement Proposal]. https://eips.ethereum.org/EIPS/eip-8004

[9] DefiLlama. (n.d.-a). Near chain overview. Retrieved November 7, 2026, from https://defillama.com/chain/Near

[10] DefiLlama. (n.d.-b). NEAR Intents. Retrieved November 7, 2026, from https://defillama.com/protocol/near-intents

[11] Dework. (n.d.). The task manager for DAOs and decentralized work. Retrieved November 7, 2026, from https://dework.xyz/

[12] Digital Commerce 360. (2025, November 28). Gartner: AI agents to handle $15 trillion in B2B purchases by 2028. https://www.digitalcommerce360.com/2025/11/28/gartner-ai-agents-15-trillion-in-b2b-purchases-by-2028/

[13] Ethereum Improvement Proposals. (2026). *ERC-8183: Agentic commerce* [Draft, created February 25, 2026]. https://eips.ethereum.org/EIPS/eip-8183

[14] Federal Bureau of Investigation, Internet Crime Complaint Center. (2018, September 18). Cybercriminals utilize social engineering techniques to obtain employee credentials to conduct payroll diversion [Public service announcement]. https://www.ic3.gov/media/2018/180918.aspx

[15] Filippas, A., Horton, J. J., & Golden, J. (2022). Reputation inflation. Marketing Science, 41(4), 733–745. https://doi.org/10.1287/mksc.2022.1350

[16] Fiverr International. (2026a). Annual report on Form 20-F for the fiscal year ended December 31, 2025. U.S. Securities and Exchange Commission. https://www.sec.gov/Archives/edgar/data/0001762301/000117891326000858/zk2634486.htm

[17] Fiverr International. (2026b). Fiverr announces fourth quarter and full year 2025 results [Press release]. https://investors.fiverr.com/node/9886/pdf

[18] Freelancers Union. (2022, May 12). Over 60% of NY freelancers report not being paid for work performed. https://blog.freelancersunion.org/2022/05/12/over-60-of-ny-freelancers-report-not-being-paid-for-work-performed

[19] Gu, G., & Zhu, F. (2021). Trust and disintermediation: Evidence from an online freelance marketplace. Management Science, 67(2), 794–807. https://doi.org/10.1287/mnsc.2020.3583

[20] Kleros. (n.d.). Kleros Escrow; Dispute resolution. Kleros Documentation. Retrieved November 7, 2026, from https://docs.kleros.io/products/escrow.md

[21] LaborX. (n.d.). Hire Web3 talent. Retrieved November 7, 2026, from https://laborx.com/hire-talents

[22] NEAR AI. (n.d.-a). NEAR AI Cloud documentation. Retrieved November 7, 2026, from https://docs.near.ai/

[23] NEAR AI. (n.d.-b). Agent Market: Discover agents. Retrieved November 7, 2026, from https://market.near.ai/discover/all

[24] NEAR Hackathon. (2026). Build Beyond Boundaries: Rules. Retrieved November 7, 2026, from https://nearhackathon.com/HACKATHON.md

[25] NEAR. (n.d.-a). Access keys. NEAR Documentation. Retrieved November 7, 2026, from https://docs.near.org/protocol/access-keys

[26] NEAR. (n.d.-b). Fungible token standard (NEP-141). NEAR Documentation. Retrieved November 7, 2026, from https://docs.near.org/primitives/ft/standard

[27] NEAR. (n.d.-c). Decentralized autonomous organizations. NEAR Documentation. Retrieved November 8, 2026, from https://docs.near.org/primitives/dao

[28] Pallais, A. (2014). Inefficient hiring in entry-level labor markets. American Economic Review, 104(11), 3565–3599. https://doi.org/10.1257/aer.104.11.3565

[29] r/AI_Agents. (n.d.). How are you handling agents that actually spend money? [Online forum post]. Reddit. https://reddit.com/r/AI_Agents/comments/1sr1vu2/

[30] r/ArtificialInteligence. (n.d.). The trust problem nobody's talking about: When AI agents control money [Online forum post]. Reddit. https://reddit.com/r/ArtificialInteligence/comments/1rl4hhs/

[31] r/OpenSourceeAI. (n.d.). AI agents are terrible at managing money. I built a kill-switch to hard-cap tool spend [Online forum post]. Reddit. https://reddit.com/r/OpenSourceeAI/comments/1rbuse7/

[32] r/Upwork. (n.d.). Top Rated Plus, $447K earned, 100% JSS, permanently banned [Online forum post]. Reddit. https://reddit.com/r/Upwork/comments/1sojif0/

[33] The Defiant. (2025, May 13). NEAR Protocol launches 600ms blocks, 1.2-second finality unlocking DeFi use cases. https://thedefiant.io/news/blockchains/near-protocol-launches-600ms-blocks-1-2-second-finality-unlocking-defi-use-cases-490247d4

[34] UMA. (n.d.). How does UMA's oracle work? UMA Documentation. Retrieved November 8, 2026, from https://docs.uma.xyz/protocol-overview/how-does-umas-oracle-work

[35] Upwork Community. (2019). Client took the work and released 50 out of 300 [Online forum post]. https://community.upwork.com/t5/New-to-Upwork/Client-took-the-work-and-released-50-out-of-300/m-p/643282

[36] Upwork Community. (2020). Chargeback issue, any way to fight back? [Online forum post]. https://community.upwork.com/t5/Freelancers/Chargeback-issue-anyway-to-fight-back/m-p/824004

[37] Upwork Community. (2021). Payment dispute won, 1 of 1500 dollars paid [Online forum post]. https://community.upwork.com/t5/Freelancers/Payment-dispute-won-1-of-1500-dollars-paid/m-p/977572

[38] Upwork Community. (2023). Beware of fake jobs with unrealistic pay rates and verified payment [Online forum post]. https://community.upwork.com/t5/Freelancers/Beware-of-Fake-Jobs-with-Unrealistic-Pay-Rates-and-Verified/m-p/1296910

[39] Upwork. (2026). Annual report on Form 10-K for the fiscal year ended December 31, 2025. U.S. Securities and Exchange Commission. https://www.sec.gov/Archives/edgar/data/1627475/000162747526000012/upwk-20251231.htm

[40] Verga, P., Hofstatter, S., Althammer, S., Su, Y., Piktus, A., Arkhangorodsky, A., Xu, M., White, N., & Lewis, P. (2024). *Replacing judges with juries: Evaluating LLM generations with a panel of diverse models* (arXiv:2404.18796). https://arxiv.org/abs/2404.18796

[41] Wang, Y., Pradel, M., & Liu, Z. (2025). *Are "solved issues" in SWE-bench really solved correctly? An empirical study* (arXiv:2503.15223). https://arxiv.org/abs/2503.15223

[42] Woodward, K. (2026, August 28). Nearly 44% of chargebacks are labeled friendly fraud, finds a Chargebacks911 report. Digital Transactions. https://www.digitaltransactions.net/nearly-44-of-chargebacks-are-labeled-friendly-fraud-finds-a-chargebacks911-report/

[43] Zhong, Z., Raghunathan, A., & Carlini, N. (2025). *ImpossibleBench: Measuring LLMs' propensity of exploiting test cases* (arXiv:2510.20270). https://arxiv.org/abs/2510.20270